Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 7

на тему «Обеспечение надёжного взаимодействия сервисов и оптимизация распределённой системы»

Выполнил: Летко А.С.
студент группы 377901

Проверила: Сильванович Ю.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы – повысить устойчивость распределённой платёжной системы к временным сбоям сетевого взаимодействия, организовать кеширование часто используемых данных в Redis и обеспечить наблюдаемость бизнес-сценариев средствами журналирования.

В работе рассматриваются GraphQL Gateway, сервис авторизации и внутренние сервисы, использующие gRPC, PostgreSQL и Kafka. Реализованы ограничение времени ожидания, ограниченные повторные попытки, обработка ошибок, кеширование справочных данных, публикация событий через outbox и защита потребителей от повторной обработки.

2 ИСХОДНАЯ СИСТЕМА И ПОСТАНОВКА ЗАДАЧИ

В качестве основы использована система предыдущей лабораторной работы. Клиент обращается к единой точке входа GraphQL Gateway, а gateway проверяет права и запрашивает данные у внутренних сервисов. Данные размещены в PostgreSQL; Kafka передаёт события об изменении состояния платежа; Redis использовался для хранения пользовательских сессий.

В распределённой системе сетевой запрос может завершиться с ошибкой даже при корректных входных данных. Зависший ответ удерживает ресурсы вызывающего сервиса, кратковременное отключение компонента делает операцию недоступной, а повторная доставка события способна привести к повторной записи. Поэтому необходимо ограничивать ожидание, отличать временные ошибки от ошибок данных и сохранять согласованность результатов.

В лабораторной работе расширена существующая интеграция: для безопасных gRPC-операций добавлены ограниченные повторные попытки, для Kafka – обработка ошибок и отдельный топик неуспешных сообщений, для повторяемых чтений – Redis-кеш. Логи отражают адресата, операцию, попытку и результат, а также идентификаторы событий и платежей.

2.1 Состав компонентов

Gateway предоставляет GraphQL API и контролирует доступ к данным. Auth Service регистрирует пользователей, проверяет учётные данные и управляет JWT-сессиями в Redis. Identity Service хранит пользователей, Merchant Service – магазины, Payment Method Service – способы оплаты, Account Service – сведения о счетах.

Payment Service отвечает за создание и изменение статуса платежей. Transaction Service регистрирует транзакции по событиям, Notification Service формирует уведомления, Analytics Service обновляет агрегированные показатели. Refund Service предоставляет ранее сохранённые возвраты.

Общий модуль service-host запускает нужную реализацию сервиса в отдельном контейнере в зависимости от SERVICE_NAME.

Сеть между контейнерами создаётся Docker Compose. PostgreSQL и Kafka используют именованные тома; файл db/init.sql создаёт схемы и демонстрационные данные при первой инициализации пустого тома базы. Это позволяет запустить учебный стенд без отдельной установки Java, Maven, PostgreSQL, Kafka и Redis на компьютере.

3 РАЗВЁРТЫВАНИЕ И ЗАПУСК СИСТЕМЫ

Контейнеры описаны в compose.yaml. Сборка Java-приложений выполняется многоэтапным Dockerfile: Maven компилирует проект и создаёт JAR, затем приложение запускается в образе Java Runtime. Gateway доступен на порту 18080 компьютера, а внутренние gRPC-вызовы направляются по именам сервисов на порт 9090 внутри сети Compose.

Для запуска используется команда docker compose up --build -d. Состояние контейнеров проверяется командой docker compose ps, а журналы – командой docker compose logs. Для PostgreSQL, Redis и Kafka определены проверки готовности. При остановке командой docker compose down именованные тома сохраняются; добавление параметра -v удаляет данные томов и требует осознанного применения.

На рисунке 3.1 показаны контейнеры распределённой системы в Docker Desktop. При проверке состояния командой docker compose ps все контейнеры находились в состоянии Up, а PostgreSQL, Redis и Kafka имели состояние healthy. Доступность контейнеров дополнительно проверялась выполнением запросов и обработкой событий.



Рисунок 3.1 - Запущенные контейнеры распределённой системы

4 ОБРАБОТКА GRPC-ВЫЗОВОВ И ОШИБОК

4.1 Ограничение времени ожидания

Gateway обращается к внутренним сервисам через GrpcServiceClient. Для операций чтения каждая попытка имеет gRPC deadline 4 секунды. Для операций изменения платежа установлен предел 45 секунд: внутри Payment Service последовательно выполняются проверки других сервисов, и каждой из них также требуется время на возможные повторы.

Сервис платежей использует класс GrpcRetry для запросов Identity, Merchant, Payment Method и Account. Каждая попытка создаётся с deadline 4 секунды. Таймаут не означает, что удалённая операция обязательно не выполнилась: ответ мог быть потерян после её завершения. Поэтому при вызовах, изменяющих состояние, автоматический повтор на gateway не применяется.

4.2 Повторные попытки

Повторы допускаются только для безопасных операций чтения и проверок. Выполняется не более трёх попыток, между ними предусмотрена возрастающая задержка 200 и 400 миллисекунд. Повторяются коды UNAVAILABLE, DEADLINE_EXCEEDED и RESOURCE_EXHAUSTED; ошибки входных данных не повторяются. При прерывании ожидания сохраняется признак прерывания потока и операция завершается ошибкой.

При создании и изменении платежа автоматический повтор отсутствует, поскольку повторная отправка может создать второй платёж или привести к неоднозначному результату. Если клиент не получил ответ, необходимо проверить состояние платежа до повторной отправки. Для промышленного применения возможен отдельный механизм ключей идемпотентности, но в данной реализации он не добавлен.

Вызовы и повторные попытки регистрируются с именем сервиса, операцией и номером попытки. После исчерпания попыток фиксируется код gRPC и вызывающей стороне возвращается ошибка. На рисунке 4.1 приведён журнал gateway при остановленном Merchant Service: видны повторные попытки и завершение обращения с кодом UNAVAILABLE. После проверки сервис был запущен повторно.

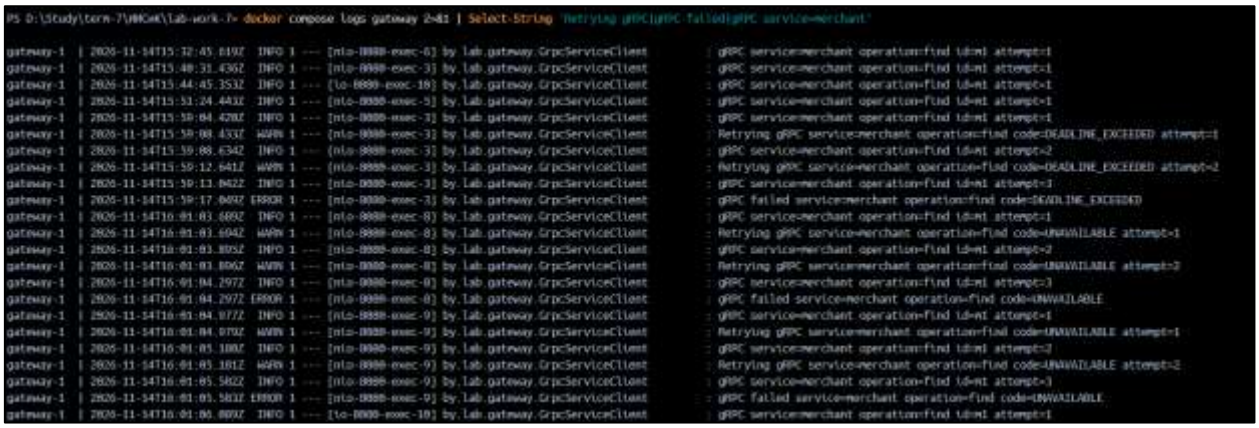


Рисунок 4.1 - Повторные попытки при недоступности gRPC-сервиса

4.3 Представление ошибок пользователю

Сервер RpcServer преобразует некорректные аргументы в статус INVALID_ARGUMENT. Ошибки сетевого взаимодействия сохраняют соответствующий статус gRPC, а неожиданные внутренние ошибки возвращаются как INTERNAL без передачи подробностей реализации клиенту. В журнале сервиса сохраняется информация об исключении.

На gateway ошибки бизнес-валидации представляются как ошибки некорректного запроса, а ошибки недоступности – как GraphQL-ошибка SERVICE_UNAVAILABLE. Отдельно проверяются полномочия пользователя. На рисунке 4.2 приведён фактически полученный ответ FORBIDDEN: клиент u1 запросил данные пользователя u2 и получил отказ, а поле user стало null.

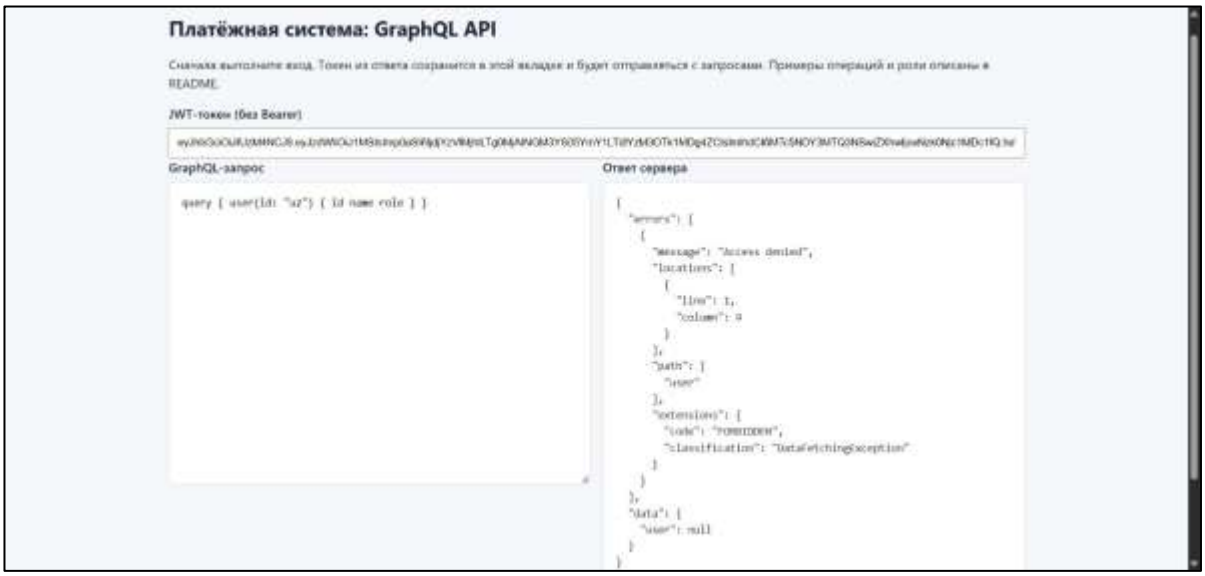


Рисунок 4.2 - Обработка отказа в доступе через GraphQL

5 КЕШИРОВАНИЕ СПРАВОЧНЫХ ДАННЫХ В REDIS

5.1 Выбор данных

В качестве объектов кеширования выбраны записи пользователей и магазинов при запросе по идентификатору. Эти данные используются повторно, в частности при разрешении вложенных полей client и merchant у платежа. Кеш располагается в gateway и снижает число одинаковых запросов к Identity Service и Merchant Service.

Ключи имеют вид reference:identity:<id> и reference:merchant:<id>. Значение хранится в JSON, время жизни составляет 30 секунд. Отсутствующие записи не кешируются. После истечения срока очередной запрос снова обращается к исходному сервису и сохраняет актуальный результат.

5.2 Порядок чтения и ограничения

Сначала gateway пытается получить значение по ключу Redis. При попадании в кеш значение преобразуется из JSON и возвращается вызывающему коду. При отсутствии ключа выполняется gRPC-запрос, а непустой ответ сохраняется в Redis. Если чтение или запись кеша завершились ошибкой, она фиксируется в журнале; данные запрашиваются у исходного сервиса, и работа с кешем не становится обязательным условием получения справочника.

Кеш имеет ограниченную актуальность: внешнее изменение пользователя или магазина может быть отражено с задержкой до 30 секунд. Операций изменения этих справочников в текущем GraphQL API нет. Данные самого платежа и его статуса не кешируются, чтобы повторное чтение после изменения состояния не получило устаревшую копию.

Redis используется также для авторизационных сессий, но эта функция отличается от кеша справочных данных. При отсутствии Redis проверка сессии не может подтвердить действительность токена, поэтому доступ к защищённым операциям не предоставляется. Таким образом, fallback кеша не означает обход авторизации.

Запрос платежа со вложенными данными клиента и магазина представлен на рисунке 5.1. На рисунке 5.2 журнал gateway показывает сохранение справочных данных в Redis строками Cache stored и последующее получение тех же данных из кеша строками Cache hit для identity u1 и merchant m1.

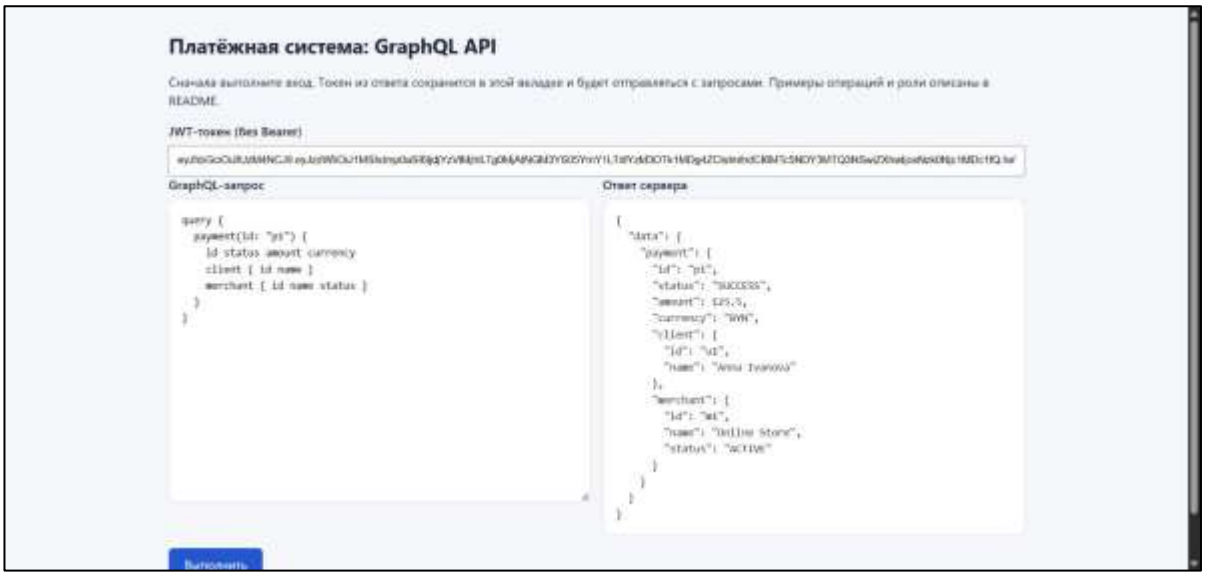


Рисунок 5.1 - Получение платежа с данными клиента и магазина

```
PS D:\Study\term-7\ИИСиК\lab-work-7> docker compose logs gateway 2>&1 | Select-String 'Cache stored|Cache hit'

gateway-1  | 2026-11-14T15:32:45.616Z  INFO 1 --- [nio-8080-exec-6] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T15:32:46.281Z  INFO 1 --- [nio-8080-exec-6] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T15:40:31.433Z  INFO 1 --- [nio-8080-exec-3] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T15:40:31.467Z  INFO 1 --- [nio-8080-exec-3] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T15:44:45.356Z  INFO 1 --- [io-8080-exec-10] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T15:44:45.373Z  INFO 1 --- [io-8080-exec-10] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T15:51:24.439Z  INFO 1 --- [nio-8080-exec-5] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T15:51:24.461Z  INFO 1 --- [nio-8080-exec-5] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T16:02:47.884Z  INFO 1 --- [nio-8080-exec-2] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T16:02:48.556Z  INFO 1 --- [nio-8080-exec-2] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T16:04:16.178Z  INFO 1 --- [nio-8080-exec-5] by.lab.gateway.GrpcServiceClient        : Cache stored service=identity id=u1
gateway-1  | 2026-11-14T16:04:16.211Z  INFO 1 --- [nio-8080-exec-5] by.lab.gateway.GrpcServiceClient        : Cache stored service=merchant id=m1
gateway-1  | 2026-11-14T16:04:17.399Z  INFO 1 --- [nio-8080-exec-4] by.lab.gateway.GrpcServiceClient        : Cache hit service=identity id=u1
gateway-1  | 2026-11-14T16:04:17.402Z  INFO 1 --- [nio-8080-exec-4] by.lab.gateway.GrpcServiceClient        : Cache hit service=merchant id=m1
gateway-1  | 2026-11-14T16:04:18.640Z  INFO 1 --- [nio-8080-exec-6] by.lab.gateway.GrpcServiceClient        : Cache hit service=identity id=u1
gateway-1  | 2026-11-14T16:04:18.643Z  INFO 1 --- [nio-8080-exec-6] by.lab.gateway.GrpcServiceClient        : Cache hit service=merchant id=m1
gateway-1  | 2026-11-14T16:04:19.521Z  INFO 1 --- [nio-8080-exec-7] by.lab.gateway.GrpcServiceClient        : Cache hit service=identity id=u1
gateway-1  | 2026-11-14T16:04:19.524Z  INFO 1 --- [nio-8080-exec-7] by.lab.gateway.GrpcServiceClient        : Cache hit service=merchant id=m1
```

Рисунок 5.2 - Запись и получение справочных данных из Redis

6 НАДЁЖНАЯ ОБРАБОТКА СОБЫТИЙ KAFKA

6.1 Запись события и публикация

Payment Service создаёт события PaymentCreated, PaymentConfirmed и PaymentCancelled. Событие содержит eventId, eventType, paymentId, clientId, merchantId, amount, currency и timestamp. Идентификаторы позволяют связать действие пользователя, изменение платежа и последующую обработку события.

Изменение платежа и добавление события в payment.outbox выполняются в одной транзакции PostgreSQL. Если бизнес-операция завершается исключением, транзакция откатывается и событие не остаётся без соответствующего изменения. Периодическая задача выбирает

неопубликованные записи, отправляет их в топик payment-events и только после подтверждения Kafka помечает запись опубликованной.

При ошибке публикации сообщение об ошибке записывается в журнал, запись остаётся в outbox и будет выбрана при следующем запуске задачи. Время блокировки отправки и ожидания доставки ограничено настройками producer. Между подтверждением Kafka и обновлением outbox возможен сбой, поэтому повторная публикация одного и того же eventId допустима и должна обрабатываться потребителями без дублирования результата.

6.2 Идемпотентность и обработчики

Transaction Service, Notification Service и Analytics Service используют собственные группы потребителей и независимо читают события. Перед бизнес-обработкой каждый записывает eventId в таблицу своей схемы processed_events. Вставка идентификатора и изменение бизнес-данных находятся в одной транзакции. Если идентификатор уже существует, событие пропускается; если транзакция завершилась ошибкой, запись идентификатора также откатывается.

При PaymentConfirmed сервис транзакций создаёт запись PAYMENT, сервис уведомлений – уведомление со статусом CREATED, а сервис аналитики увеличивает количество и сумму платежей за день для магазина и валюты. При PaymentCancelled создаётся уведомление. Событие PaymentCreated учитывается как обработанное, но не создаёт перечисленные итоговые записи.

6.3 Ошибки обработки и DLT

Kafka listener использует DefaultErrorHandler. В конфигурации задано до трёх повторов с интервалом 3 секунды. Если обработать сообщение не удалось, DeadLetterPublishingRecoverer направляет его в топик payment-events.DLT. При ошибке отправки в DLT восстановление не считается успешным. Сведения об исходном топике, разделе, смещении и ключе фиксируются в журнале.

DLT позволяет изолировать проблемное сообщение и продолжить обработку последующих записей; исправление причины и возврат сообщения на повторную обработку остаются отдельной административной операцией. На рисунке 6.1 показана проверка с некорректным тестовым сообщением: журнал содержит три неудачные попытки обработки и сообщение об исчерпании повторов, а чтение DLT подтверждает доставку тестового сообщения. Его получили независимые группы потребителей, поэтому в DLT присутствуют три копии.

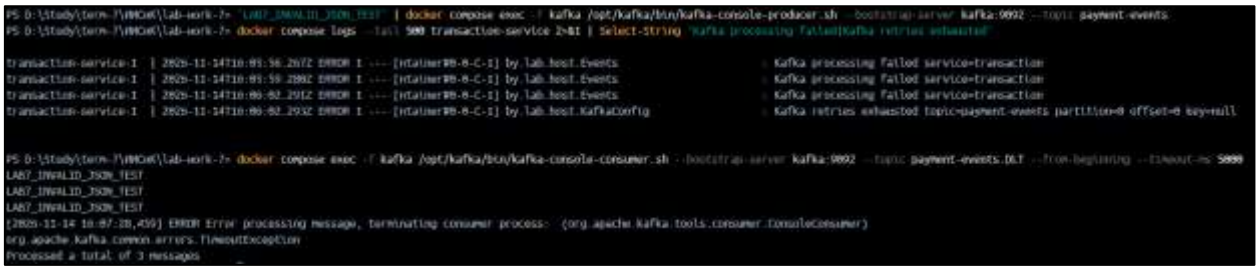


Рисунок 6.1 - Обработка ошибки Kafka и направление сообщения в DLT

7 БИЗНЕС-СЦЕНАРИИ ПЛАТЁЖНОЙ СИСТЕМЫ

7.1 Создание платежа

Для создания платежа клиент передаёт идентификаторы клиента, магазина и способа оплаты, сумму, валюту и необязательное описание. Gateway проверяет JWT и право клиента действовать от собственного имени. Payment Service проверяет обязательные поля, положительную сумму с точностью до двух знаков и трёхбуквенный код валюты.

Далее через gRPC проверяются существование пользователя с ролью CLIENT, активность магазина и принадлежность активного способа оплаты указанному клиенту. Только после успешных проверок создаётся запись со статусом CREATED и событие PaymentCreated в outbox. При ошибке проверки операция завершается без создания платежа.

7.2 Подтверждение и отмена

Подтвердить доступный платёж может продавец соответствующего магазина или администратор. Payment Service проверяет, что текущий статус – CREATED, повторно проверяет способ оплаты и обращается к Account Service за проверкой счёта, валюты и достаточности баланса. После успешных проверок статус меняется на SUCCESS, а событие PaymentConfirmed записывается в outbox.

Отменить платёж в состоянии CREATED может его клиент или администратор. При отмене устанавливается статус CANCELLED и создаётся событие PaymentCancelled. Условное обновление по статусу CREATED не позволяет двум параллельным операциям одновременно перевести один платёж из исходного состояния в разные конечные состояния.

Реализация учебная: проверка достаточности средств не сопровождается реальным списанием с клиентского счёта и зачислением магазину. Создание новых возвратов также не реализовано: API предоставляет чтение существующих записей. Указанные ограничения необходимо учитывать при оценке полноты платёжного процесса.

7.3 Получение данных и аналитики

Клиент может получить принадлежащие ему платежи, способы оплаты, связанные транзакции и возвраты. Продавец получает доступ к операциям своего магазина, администратор обладает более широкими правами. Для списка платежей предусмотрены фильтры и сортировка, а операция paymentSummary возвращает количество и сумму в одной валюте.

На рисунке 7.1 представлен фактический ответ на запрос списка и сводки. Получены два платежа p1 и p2 на общую сумму 170,5 BYN. Результат подтверждает чтение данных и подсчёт по доступной клиенту выборке; он не является проверкой записи нового платежа.

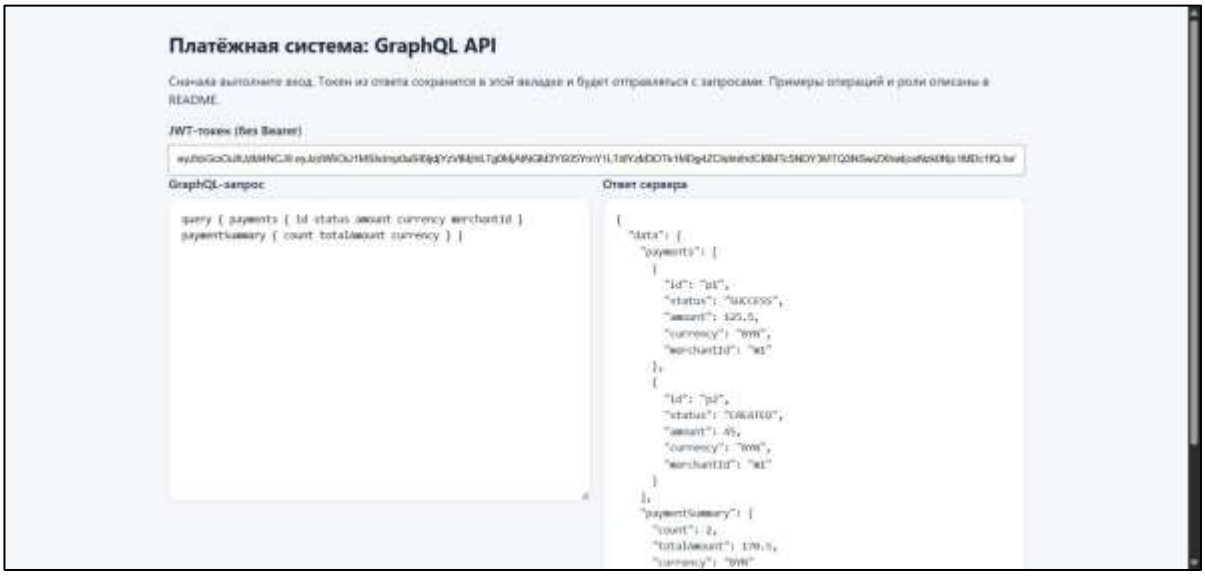


Рисунок 7.1 - Получение списка платежей и сводной суммы

На рисунке 7.2 показан фактически выполненный запрос способа оплаты клиента и транзакций платежа p1. В ответе присутствуют активная карта pm1 и две демонстрационные транзакции типов PAYMENT и REFUND. Эти записи находятся в исходных данных и не подтверждают обработку нового события Kafka во время подготовки отчёта.

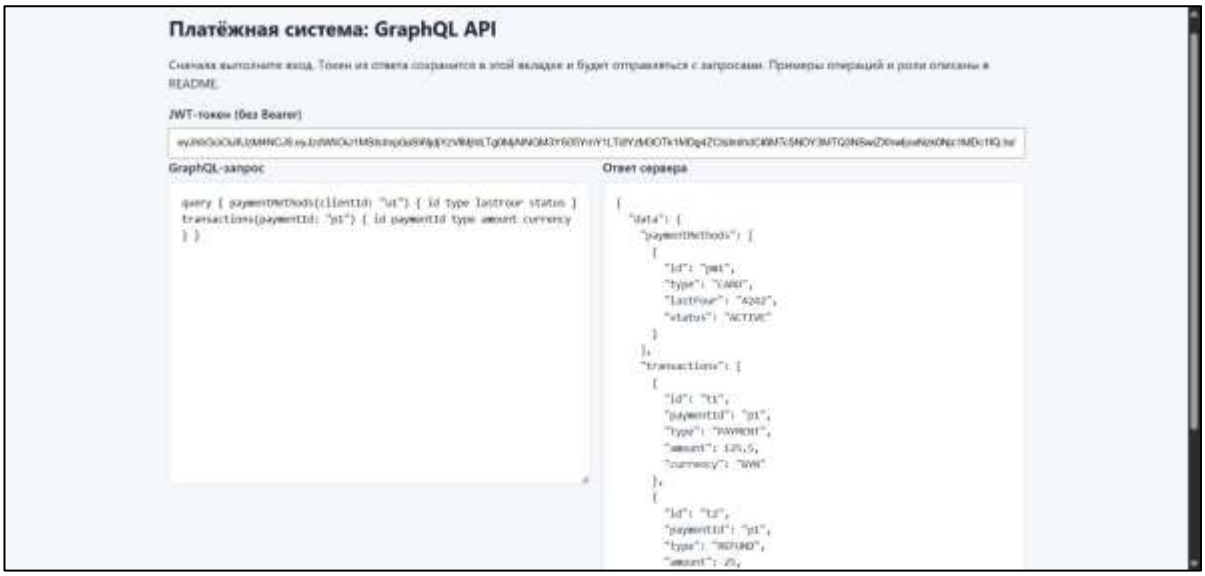


Рисунок 7.2 - Способ оплаты и транзакции платежа

8 ЛОГИРОВАНИЕ И НАБЛЮДЕНИЕ

В gateway журналируются gRPC-вызовы: сервис, операция, идентификатор, номер попытки, временная ошибка и окончательный код сбоя. При работе Redis регистрируются попадание в кеш, сохранение и ошибки чтения или записи. Это позволяет определить, был ли запрос обслужен кешем или направлен во внутренний сервис.

В сервисах журналируются входящие gRPC-запросы и ошибки, создание и смена статуса платежа. Для событий указываются идентификатор события, идентификатор платежа, тип и имя сервиса-потребителя. Повторная доставка отмечается отдельно, что позволяет отличать её от нового бизнес-события.

Для просмотра связанных операций удобно открыть docker compose logs -f gateway payment-service transaction-service notification-service analytics-service. Журнал outbox показывает публикацию события или ошибку, после которой запись будет обработана повторно. Для разбирательства следует сопоставлять paymentId и eventId и учитывать асинхронную задержку между сменой статуса и обработкой потребителями.

На рисунке 8.1 показан журнал учебного платежа: Payment Service зафиксировал переход в состояние SUCCESS и публикацию события PaymentConfirmed. Одинаковый eventId в журналах Transaction Service, Notification Service и Analytics Service связывает публикацию с последующей обработкой.



Рисунок 8.1 - Журнал выполнения распределённого сценария

9 ПРОВЕРКА СИСТЕМЫ И АНАЛИЗ РЕЗУЛЬТАТОВ

После развёртывания проекта проверено состояние контейнеров командой docker compose ps: компоненты находятся в состоянии Up, проверки готовности инфраструктурных сервисов успешны. В веб-интерфейсе выполнен вход демонстрационного клиента anna@example.com. Запрос платежа p1 вернул статус SUCCESS и вложенные данные клиента и магазина.

Также выполнены запрос списка платежей и сводной суммы, получение способов оплаты и транзакций, а также запрос недоступного клиенту пользователя. Последний вернул FORBIDDEN. Полученные ответы показаны на рисунках 4.2, 5.1, 7.1 и 7.2. Журнал gateway подтвердил как сохранение данных в кеш, так и их последующее чтение из кеша.

При остановленном Merchant Service проверены повторные gRPC-вызовы и итоговая обработка недоступности, после чего сервис был запущен обратно. Для проверки Kafka в payment-events отправлено некорректное тестовое сообщение: журнал потребителя зафиксировал неудачные попытки, а чтение payment-events.DLT подтвердило помещение сообщения в DLT. Отдельное восстановление после отказа Kafka не проверялось.

Создан и подтверждён учебный платёж на 10 BYN. По идентификаторам платежа и события в журналах прослежены публикация PaymentConfirmed и его получение сервисами транзакций, уведомлений и аналитики. Результаты проверки приведены на рисунке 8.1.

10 ЗАКЛЮЧЕНИЕ

В ходе работы существующая распределённая платёжная система дополнена ограничением времени ожидания и ограниченными повторными попытками безопасных межсервисных вызовов. Ошибки gRPC преобразуются в контролируемые ответы, а журналы позволяют определить сервис, операцию и этап возникновения проблемы.

Организовано кеширование справочных данных пользователей и магазинов в Redis с ограниченным временем жизни и обращением к

исходному сервису при ошибке кеша. Для асинхронного обмена сохранён транзакционный outbox, обеспечена идемпотентность потребителей и добавлена обработка неуспешных сообщений через DLT.

На запущенном стенде проверены вход клиента, чтение платежей, кеширование, обработка отказа gRPC и ошибочного сообщения Kafka, а также публикация и получение события подтверждения платежа. Результаты показывают работу реализованных механизмов на рассмотренных сценариях; выполнение реального перевода денег и создание возврата в системе не реализованы.